Exercise 2 — Job Workers with the Camunda Spring SDK

> **Goal:** add the Spring-based job workers that drive the deterministic service tasks of the Office Alpaca process — `check-office-readiness` and `schedule-alpaca-visit`.

What you will learn

- How the Camunda Spring SDK 8.9 wires Java methods to BPMN service tasks via `@JobWorker`.
- How to fetch a small set of process variables with `@Variable` and POJO mapping.
- How worker config (timeouts, retries, max active jobs) is layered between BPMN and `application.yaml`.

Prerequisites

1. Java 21+ and Maven installed (`java -version` , `mvn -version`).
2. Clone the repo: https://github.com/McAlm/blank-demo
3. A Camunda 8 SaaS cluster you can deploy to.

The big picture

Your BPMN has two service tasks that are intentionally *not* connectors — they exist precisely so attendees can implement them as job workers and see the pattern repeat:

BPMN element	`taskDefinition.type`	Worker class
Check office readiness	`check-office-readiness`	`OfficeReadinessWorker`
Schedule alpaca visit	`schedule-alpaca-visit`	`ScheduleVisitWorker`

A worker = a Spring bean with one or more `@JobWorker` annotated methods. The Camunda Spring SDK auto-detects them, starts a polling/streaming loop and routes activated jobs to your method.

Step 1 — Configure the SaaS connection

Create empty `src/main/resources/application.yaml` : Create an API Key in your SaaS cluster, copy & paste the Springboot yaml configuration to your application.yaml file

Step 2 — Verify the SaaS connection

Run the blank application without any further changes with

```
mvn spring-boot:run
```

You should see in the console that a process "DoNothing" has been deployed, a process instance has started and a JobWorker was triggered.

Step 3 — Write the OfficeReadinessCheck worker

Create `OfficeReadinessWorker.java` .

```
@JobWorker(type = "check-office-readiness", , fetchAllVariables = false,
fetchVariables = {"visitDate", "officeLocation"})
public Map<String, Object> checkOfficeReadiness(@Variable String
visitDate, @Variable String officeLocation) {
    // implement some basic logic
    // boolean officeReady = ...
    // return Map.of("officeReady", officeReady);
}
```

Key points:

- `type` **must** equal the `zeebe:taskDefinition type="..."` in the BPMN. Case-sensitive.
- Returning `Map.of("officeReady", ...)` adds new variables to the process scope.

Step 4 — Write the OfficeReadinessCheck worker with a `@VariablesAsType` POJO

```
@JobWorker(type = "schedule-alpaca-visit")
public ScheduleVisitResponse scheduleVisit(@VariablesAsType
ScheduleVisitRequest request) {
  //implement the -Request and Response POJOs
  // -Request contains visitDate and officeLocation
  // -Response contains confirmedVisitDate and confirmedOfficeLocation
  // just log the request data
  //return new response data
  ScheduleVisitResponse svResp = ...;
  return svr;
 }
```

`@VariablesAsType` fetches *only the fields declared on the POJO* — useful when the POJO is the entire input.

Step 6 — Deploy and start

```
mvn spring-boot:run
```

You should see log lines like:

```
Configuring 2 Job worker(s) of bean 'officeReadinessWorker',
'scheduleVisitWorker', ...
```

Acceptance criteria

- [ ] All two worker classes start without errors when running `mvn spring-boot:run`.
- [ ] Operate shows the workers consuming jobs and producing variables.

Reference links

- Camunda Spring Boot Starter — Getting started
- Job worker configuration
- React to problems with `CamundaError`
- Job worker concepts